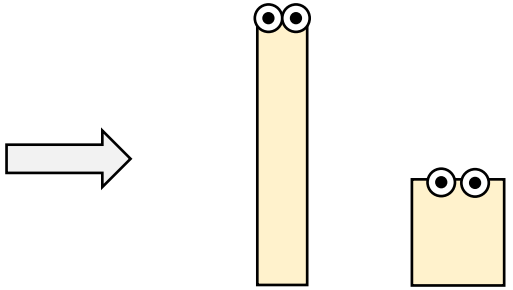
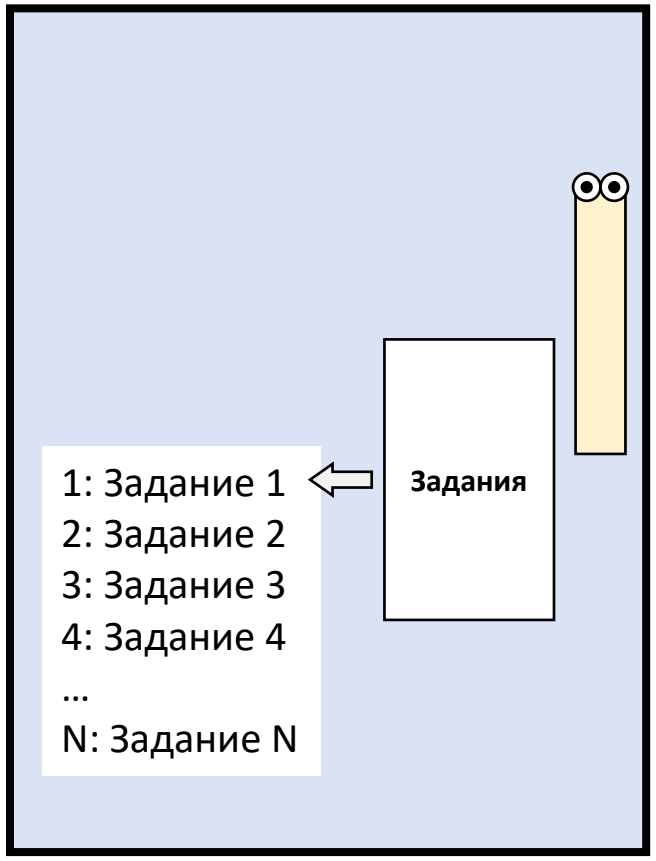
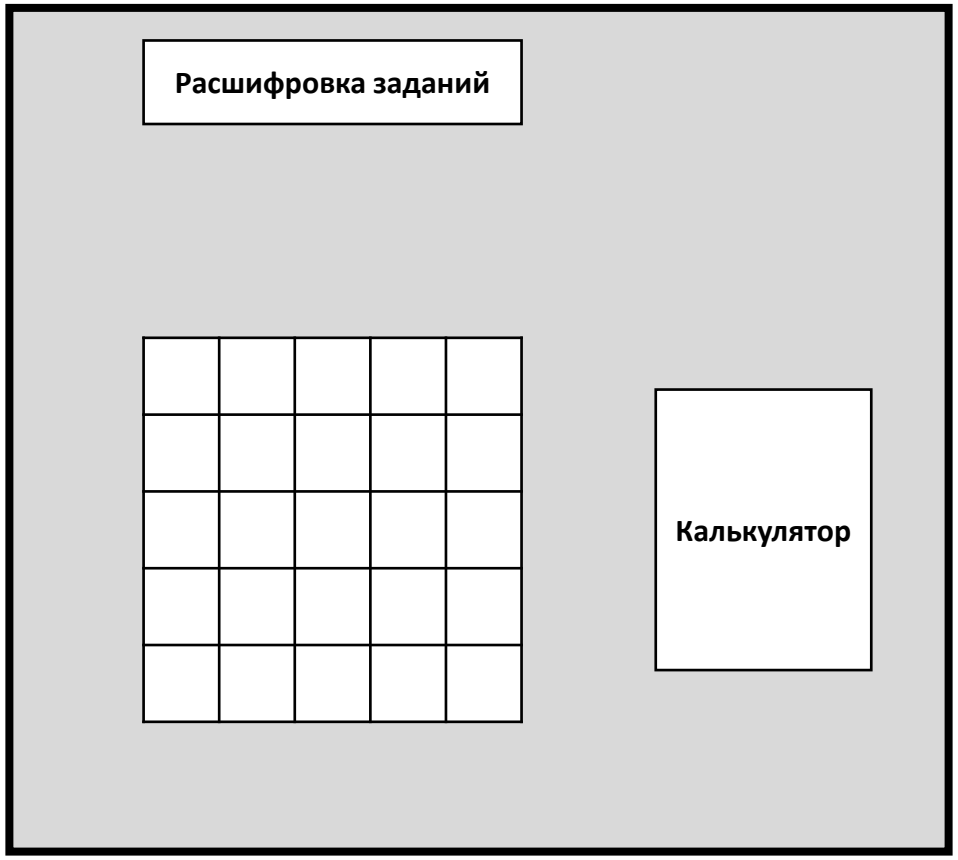
# Ассемблер

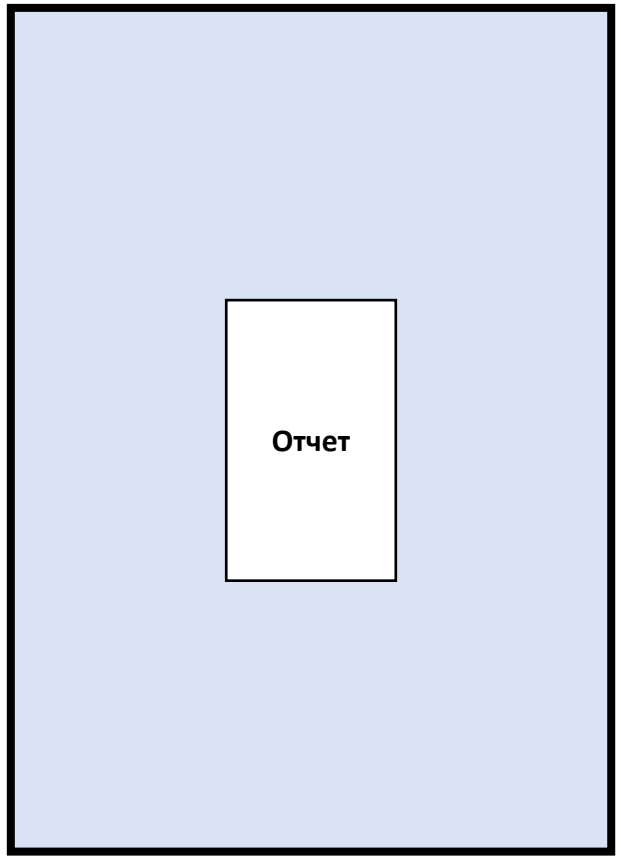
# Кто живет в компьютере?



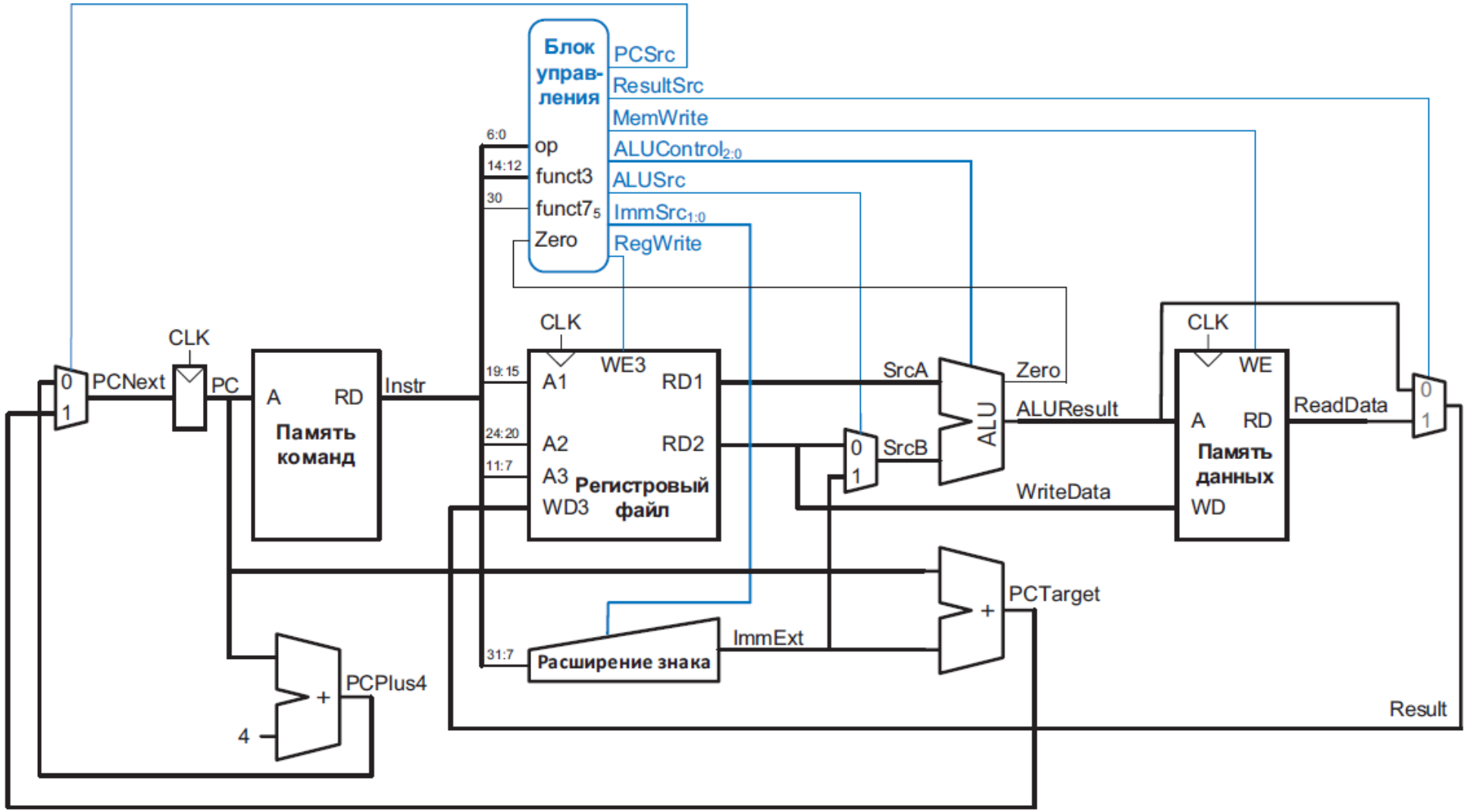
# Как живут в компьютере?



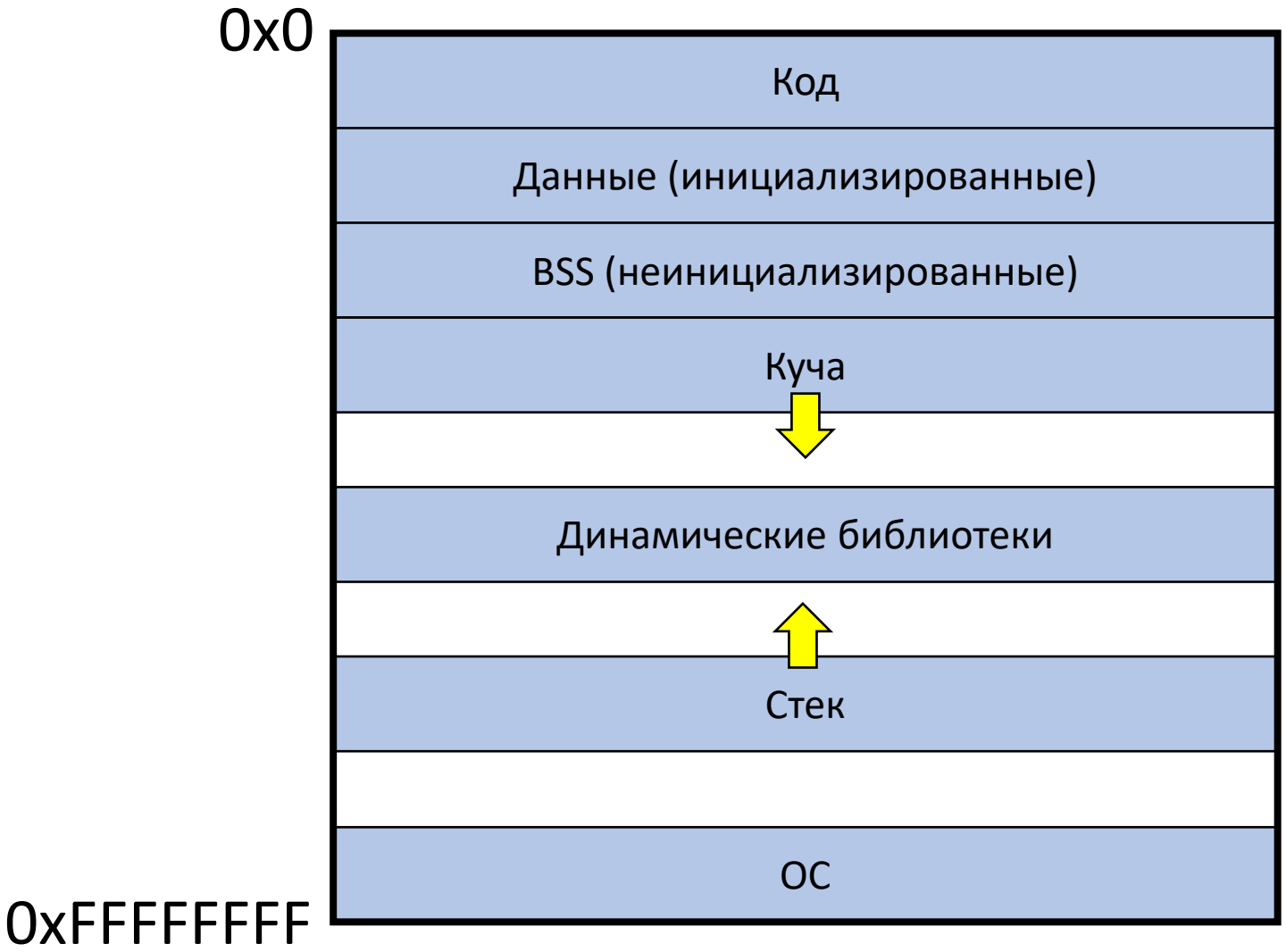



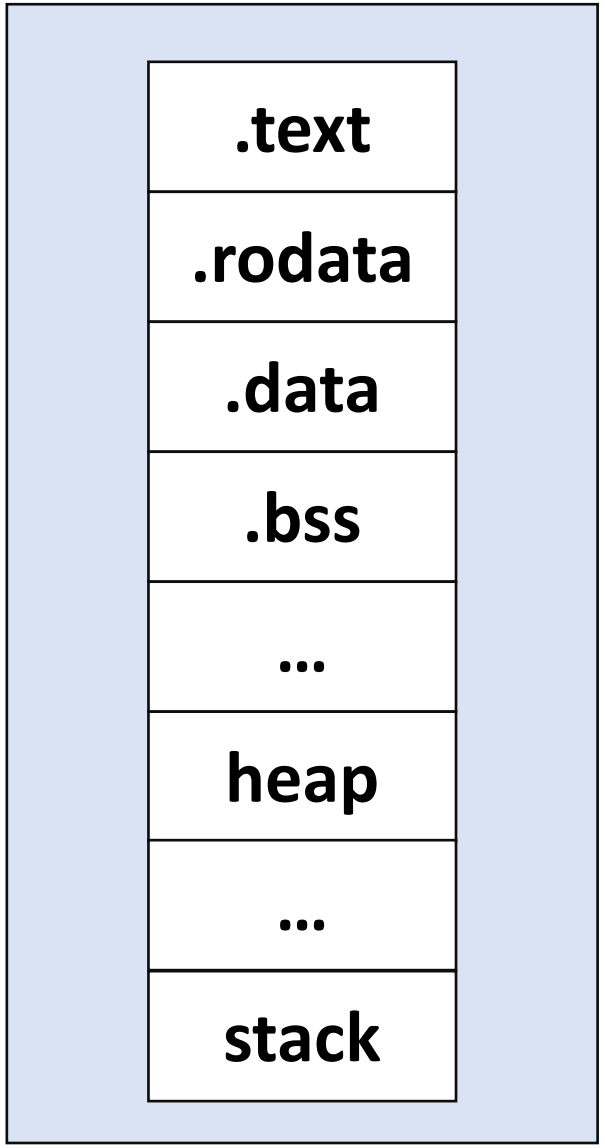
# Схема однотактного процессора



# Секции программы



# Секции программы




Код программы

Глобальная константы

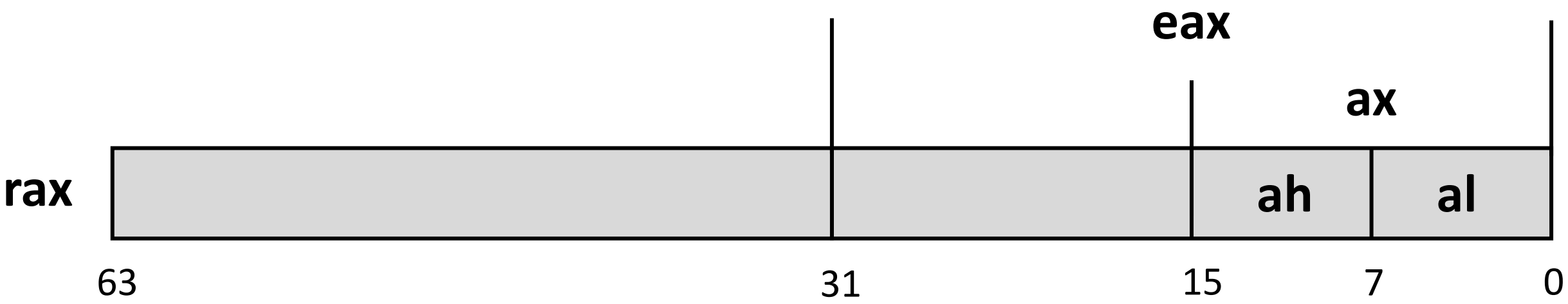
Инициализированная глобальные переменные

Неинициализированная глобальные переменные

Куча

Стек

# Регистры



# Кратко об ассемблере

| Регистр | Назначение |
|---|---|
| %eax | хранение результатов промежуточных вычислений |
| %ebx | хранения адреса (указателя) на некоторый объект в памяти |
| %ecx | счетчик |
| %edx | хранения результатов промежуточных вычислений |
| %esp | содержит адрес вершины стека |
| %ebp | указатель базы кадра стека |
| %esi | индекс источника |
| %edi | индекс приёмника |

| Команда | Назначение |
|---|---|
| **mov** *источник, назначение* | копирование *источника* в *назначение* |
| **lea** *источник, назначение* | помещает адрес *источника* в *назначение* |
| **add** *источник*, приёмник | *приёмник* = *приёмник* + *источник* |
| **sub** *источник*, приёмник | *приёмник* = *приёмник* - *источник* |
| **push** *источник* | поместить в стек |
| **pop** *назначение* | извлечь из стека |
| **cmp** *операнд_2, операнд_1* | *операнд_1 – операнд_2* и устанавливает флаги |
| **jle** *метка* | Переход если <= |

# Регистры

| 64-bit register | Lowest 32-bits | Lowest 16-bits | Lowest 8-bits |
|---|---|---|---|
| rax | eax | ax | al |
| rbx | ebx | bx | bl |
| rcx | ecx | cx | cl |
| rdx | edx | dx | dl |
| rsi | esi | si | sil |
| rdi | edi | di | dil |
| rbp | ebp | bp | bpl |
| rsp | esp | sp | spl |

# Системные вызовы

```
#
# 64-bit system call numbers and entry vectors
#
# The format is:
# <number> <abi> <name> <entry point>
#
# The __x64_sys_*() stubs are created on-the-fly for sys_*() system calls
#
# The abi is "common", "64" or "x32" for this file.
#
0       common  read                    sys_read
1       common  write                   sys_write
2       common  open                    sys_open
3       common  close                   sys_close
```

https://github.com/torvalds/linux/blob/master/**arch/x86/entry/syscalls/syscall_64.tbl**

# Системные вызовы

| %rax | System call | %rdi | %rsi | %rdx |
|---|---|---|---|---|
| 0 | sys_read | unsigned int fd | char *buf | size_t count |
| 1 | sys_write | unsigned int fd | const char *buf | size_t count |
| 2 | sys_open | const char *filename | int flags | int mode |
| 3 | sys_close | unsigned int fd | | |

https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/

# Классический пример

```
#include <unistd.h>

int main()
{
    char str[] = "Hello, MIET!\n";
    write(1, str, sizeof(str) - 1);
    _exit(0);
}
```

Вместо стандартных функций мы используем напрямую системные вызовы

# Классический пример

```
 .global _start
        .text
_start:
        # write(1, message, 12)
        mov     $1, %rax                # системный вызов 1
        mov     $1, %rdi                # связываем с файловым дескриптором 1
        mov     $message, %rsi          # передаем адрес строки
        mov     $12, %rdx               # размер строки
        syscall                         # системный вызов

        # exit(0)
        mov     $60, %rax               # системный вызов 60
        mov     $0, %rdi                # передаем 0 в качестве аргумента
        syscall                         # системный вызов
message:
        .ascii  "Hello, MIET\n"
```

```
gcc -c ex1.s
ld ex1.o -o ex1
./ex1
```

Проанализируем программу -

```
readelf -e ex1
```

# Классический пример

```
 .global _start
        .text
_start:
        # write(1, message, 12)
        mov     $1, %rax                # системный вызов 1
        mov     $1, %rdi                # связываем с файловым дескриптором 1
        mov     $message, %rsi          # передаем адрес строки
        mov     $12, %rdx               # размер строки
        syscall                         # системный вызов

        # exit(0)
        mov     $60, %rax               # системный вызов 60
        mov     $0, %rdi                # передаем 0 в качестве аргумента
        syscall                         # системный вызов

       .data
message:
        .ascii  "Hello, MIET\n"
```

```
gcc -c ex1.s && ld ex1.o -o ex1 && readelf -e ex1
```

Task 090

# Данные

.byte — размещает каждое выражение как 1 байт;
.short — 2 байта;
.long — 4 байта;
.quad — 8 байт;
.ascii — разместить строку без нуль символа;
.string — разместить строку с нуль символом;

# Самостоятельное задание

| %rax | System call | %rdi | %rsi | %rdx |
|---|---|---|---|---|
| 0 | sys_read | unsigned int fd | char *buf | size_t count |
| 1 | sys_write | unsigned int fd | const char *buf | size_t count |
| 2 | sys_open | const char *filename | int flags | int mode |
| 3 | sys_close | unsigned int fd | | |

Добавьте ввод сообщение с помощью системного вызова read

# Возможное решение

```
 .global _start
        .text
_start:
        mov     $0, %rax                # системный вызов 0
        mov     $0, %rdi                # связываем с файловым дескриптором 0
        mov     $x, %rsi                # передаем адрес строки
        mov     $5, %rdx                # размер строки
        syscall                         # системный вызов

        mov     $1, %rax                # системный вызов 1
        mov     $1, %rdi                # связываем с файловым дескриптором 1
        mov     $x, %rsi                # передаем адрес строки
        mov     $5, %rdx                # размер строки
        syscall                         # системный вызов

        mov     $60, %rax               # системный вызов 60
        mov     $0, %rdi                # передаем 0 в качестве аргумента
        syscall                         # системный вызов

        .data
x:      .long    10
```

# Обращение по адресу к глобальной переменной

```
.section .data
var:
    .quad 42


.section .text
.global _start


_start:
    lea var(%rip), %rax      # rax = &var
    mov (%rax), %rdi         # rdi = *(&var) = 42
    mov $60, %rax            # sys_exit
    syscall
```

```
Disassembly of section .text:

0000000000401000 <_start>:
  401000:       48 8d 05 f9 0f 00 00    lea    0xff9(%rip),%rax
  401007:       48 8b 38                mov    (%rax),%rdi
  40100a:       48 c7 c0 3c 00 00 00    mov    $0x3c,%rax
  401011:       0f 05                   syscall

Disassembly of section .data:

0000000000402000 <var>:
  402000:       2a 00                   sub    (%rax),%al
  402002:       00 00                   add    %al,(%rax)
  402004:       00 00                   add    %al,(%rax)
```

Мы знаем $\Delta$ между var и текущей инструкцией. Подставим её в команду.

Теперь нам неважно какой будет адрес текущей инструкции – относительно него всегда найдем адрес переменной

Task 091

# Ветвление

## jcc метка

| Мнемоника | Английское слово | Смысл | Тип операндов |
|---|---|---|---|
| e | equal | равенство | любые |
| n | not | инверсия условия | любые |
| g | greater | больше | со знаком |
| l | less | меньше | со знаком |
| a | above | больше | без знака |
| b | below | меньше | без знака |

# Пример ветвления

```
_start:
        movq  $6, %rax
        cmp   $5, %rax
        jne N_EQ
        #Действие 1

N_EQ:
        #Действие 2
```

Модернизируйте программу, следующим образом:

```
if(x==5)
    {
        #Вывод на экран «equal»
    }
        #Выход из программы
```

Task 092

# Циклы

```
_start:

        movq     $0, %rbx
N_EQ:
       #Действие

        add     $1, %rbx
        cmp     $5, %rbx
        jne N_EQ
```

Модернизируйте программу, следующим образом:

```
eax=0;
do
{
    #Вывод на экран «no equal»
    eax = eax + 1;
}
while(eax != 5)
#Вывод на экран «equal»
#Выход из программы
```

Task 093

# Стек

```
_start:
    # Положим число 42 в стек
    mov $42, %rax
    push %rax


    # Достанем число из стека
    pop %rbx
```

Модернизируйте программу, следующим образом:

По очереди в стек помещались числа от 1 до 10, далее достаньте их по очереди

Task 094

# Адресация и стек

```
_start:
    mov $10, %rax
    push %rax
    mov $32, %rax
    push %rax

    mov (%rsp), %rbx
    mov 8(%rsp), %rcx
```

Модернизируйте программу, следующим образом:

Измените программу следующем образом: Поместите на стек значения 10 и 2.

Далее после их помещения на стек, выполните команду pop %rax

Используя адресацию, получите значения из стека и вычтите одно из другого. Результат поместите в код возврата

Task 095

# Ассемблерные вставки

```
#include <stdio.h>

int main()
{
    int sum = 0, x = 1, y = 2;
    asm("add %1, %0"
    : "=r" (sum)
    : "r" (x), "0" (y)); // sum = x + y;
    printf("sum = %d, x = %d, y = %d \n", sum, x, y); // sum = 3, x = 1, y = 2
    return 0;
}
```

Task 097